

Skoola: Comprehensive Folder Structure Blueprint

Author: Manus AI

Date: June 26, 2025

Version: 1.0

Executive Summary

This document outlines the comprehensive folder structure for the Skoola educational software platform. Designed to support a large-scale, microservices-based architecture with extensive AI integration, multiple frontend applications, and robust administrative tools, this structure prioritizes scalability, maintainability, and developer efficiency. It adheres to industry best practices for monorepos and distributed systems, ensuring clear separation of concerns while facilitating seamless collaboration across development teams.

The proposed structure aims to:

- **Organize Code Logically:** Group related components and services together.
 - **Enhance Maintainability:** Make it easy to locate, understand, and modify code.
 - **Facilitate Scalability:** Support the addition of new features, services, and applications without disrupting existing structures.
 - **Improve Developer Onboarding:** Provide a clear and intuitive layout for new team members.
 - **Support CI/CD:** Align with automated testing, building, and deployment pipelines.
 - **Manage Dependencies:** Clearly define and isolate dependencies for each service and application.
-

Table of Contents

1. Top-Level Structure
2. `apps/` (Frontend Applications)
3. `services/` (Backend Microservices)
4. `libs/` (Shared Libraries and Utilities)
5. `ai/` (AI Models and Infrastructure)

6. `infra/` (Infrastructure as Code)
 7. `docs/` (Documentation)
 8. `scripts/` (Helper Scripts)
 9. `config/` (Global Configurations)
 10. `data/` (Data Management)
 11. `tools/` (Development Tools)
 12. `tests/` (Platform-wide Tests)
 13. Conclusion
-

1. Top-Level Structure

At the root level, the Skoola project will adopt a monorepo approach, which centralizes code for multiple applications and services within a single repository. This strategy offers benefits such as simplified dependency management, easier code sharing, and atomic commits across related components. The top-level directories are designed to provide a high-level overview of the entire platform.

```
skoola-platform/
├── apps/           # Frontend applications (Web, Mobile, Admin UI)
├── services/       # Backend microservices (APIs, business logic)
├── libs/           # Shared libraries, components, and utilities
├── ai/             # AI models, training data, and inference services
├── infra/          # Infrastructure as Code (IaC) definitions
├── docs/           # Project documentation, ADRs, design docs
├── scripts/        # Helper scripts for development, CI/CD, ops
├── config/         # Global configurations and environment settings
├── data/           # Data schemas, migrations, seed data
├── tools/          # Development tools and configurations
├── tests/          # End-to-end, integration, and performance tests
├── .github/        # GitHub Actions workflows (or .gitlab/ for GitLab CI)
├── .vscode/        # VS Code workspace settings and recommendations
├── .gitignore       # Git ignore file
├── README.md       # Project README
├── package.json    # Monorepo tooling (e.g., Lerna, Nx) configuration
└── Dockerfile      # Base Dockerfile for common build environments
```

This structure provides a clear separation between different types of components, making it easy for developers to navigate and understand the project's overall architecture. Each top-level directory serves a specific purpose, as detailed in the following sections.

2. `apps/` (Frontend Applications)

This directory will house all user-facing applications, each potentially built with different frontend frameworks or targeting different platforms (web, mobile, desktop). Each application within `apps/`

will be an independent project with its own `package.json` (or equivalent for native apps), dependencies, and build process.

```
skoola-platform/
├── apps/
│   ├── web-app/                # Main student/teacher web application (e.g., React, Next.js)
│   │   ├── public/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── pages/
│   │   │   ├── hooks/
│   │   │   ├── styles/
│   │   │   └── utils/
│   │   ├── .env.example
│   │   ├── package.json
│   │   └── tsconfig.json
│   ├── mobile-app/            # Cross-platform mobile application (e.g., React Native,
Flutter)                        Flutter)
│   │   ├── android/
│   │   ├── ios/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── screens/
│   │   │   ├── navigation/
│   │   │   └── assets/
│   │   ├── package.json
│   │   └── tsconfig.json
│   ├── admin-dashboard/       # Platform administration UI (e.g., React, Angular, Vue)
│   │   ├── public/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── views/
│   │   │   └── services/
│   │   ├── package.json
│   │   └── tsconfig.json
│   └── business-portal/       # Business/Organization specific portal (e.g., React, Svelte)
│       ├── public/
│       ├── src/
│       │   ├── components/
│       │   ├── pages/
│       │   └── api/
│       ├── package.json
│       └── tsconfig.json
```

Each application within `apps/` is treated as a distinct deployable unit, allowing for independent development, testing, and deployment cycles. This modularity is crucial for managing the complexity of a multi-faceted platform like Skoola.

3. `services/` (Backend Microservices)

This directory contains all backend microservices, each responsible for a specific business capability. This microservices architecture promotes loose coupling, independent deployment, and technology diversity. Each service will typically have its own codebase, dependencies, and API definitions.

```

skoola-platform/
├── services/
│   ├── auth-service/           # User authentication and authorization
│   │   ├── src/
│   │   │   ├── controllers/
│   │   │   ├── models/
│   │   │   ├── routes/
│   │   │   └── services/
│   │   ├── Dockerfile
│   │   ├── package.json (or requirements.txt for Python)
│   │   └── README.md
│   ├── user-profile-service/   # User profile management
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── course-management-service/ # Course creation, updates, and content organization
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── learning-progress-service/ # Tracking student progress, assessments
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── payment-service/       # Handles all payment processing and subscriptions
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── notification-service/   # Manages all platform notifications (email, push, in-app)
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── document-storage-service/ # Manages 'My Stationery' document uploads and access
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── analytics-service/      # Collects, processes, and exposes platform analytics data
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── moderation-service/     # Content moderation and flagging
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   ├── community-service/      # Forums, study groups, social features
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── package.json
│   └── api-gateway/            # Central entry point for all frontend applications
│       ├── src/
│       ├── Dockerfile
│       └── package.json

```

Each microservice is designed to be independently deployable and scalable. This structure allows different teams to work on different services concurrently, using the most appropriate technology stack for each service (e.g., Node.js for real-time services, Python for AI-heavy services, Go for high-performance services).

4. `libs/` (Shared Libraries and Utilities)

This directory is dedicated to housing reusable code, components, and utilities that are shared across multiple applications or services. This prevents code duplication and ensures consistency across the platform.

```

skoola-platform/
└─ libs/
   └─ ui-components/           # Reusable UI components (e.g., React components, design
system tokens)
   │   └─ src/
   │   └─ package.json
   │   └─ README.md
   └─ common-utils/           # General utility functions (e.g., date formatting,
validation)
   │   └─ src/
   │   └─ package.json
   │   └─ README.md
   └─ api-clients/            # Generated API clients for inter-service communication
   │   └─ src/
   │   └─ package.json
   │   └─ README.md
   └─ types/                  # Shared TypeScript/OpenAPI type definitions
   │   └─ src/
   │   └─ package.json
   │   └─ README.md
   └─ auth-sdk/               # SDK for integrating with the auth-service
   │   └─ src/
   │   └─ package.json
   │   └─ README.md

```

Managing shared libraries within a monorepo simplifies versioning and ensures that all consumers are using compatible versions of the shared code. This is critical for maintaining consistency and reducing integration issues.

5. ai/ (Open-Source AI Models and Infrastructure)

This directory is central to Skoola's AI-first approach, containing all components related to open-source AI model development, training, deployment, and inference. It separates AI-specific assets and code from general application logic, with a strong emphasis on leveraging community-driven solutions.

```
skoola-platform/  
└─ ai/  
    └─ models/                                # Trained open-source AI models (e.g., from Hugging Face,  
custom trained)  
        │  
        ├── recommendation-engine/  
        │   ├── lightfm_model.pkl  
        │   └── gorse_config.yaml  
        ├── nlp-moderation/  
        │   ├── transformers_classifier.pt  
        │   └── spacy_pipeline.json  
        └─ adaptive-learning/  
            ├── custom_rl_agent.h5  
            └── educational_data_mining_models/  
└─ training-scripts/                          # Scripts for data preprocessing and open-source model  
training/fine-tuning  
    ├── recommendation_trainer.py  
    ├── moderation_trainer.py  
    └── adaptive_learning_trainer.py  
└─ data-pipelines/                            # ETL pipelines for AI data ingestion and feature engineering  
(e.g., using Apache Spark)  
    ├── feature_store_etl.py  
    └── data_ingestion_pipeline.py  
└─ inference-services/                        # Microservices for serving open-source AI models (e.g.,  
FastAPI, Flask with ONNX Runtime)  
    ├── recommendation-api/  
    │   ├── src/  
    │   ├── Dockerfile  
    │   └── requirements.txt  
    ├── moderation-api/  
    ├── chatbot-api/                          # Using Rasa or open-source LLMs  
    └── search-api/                           # Using Meilisearch, txtai, or Typesense  
└─ notebooks/                               # Jupyter notebooks for experimentation and analysis with  
open-source tools  
    ├── data_exploration.ipynb  
    └── model_evaluation.ipynb  
└─ research/                                # Research papers, findings, and experimental code (focus on  
open-source research)  
    └─ mlo-ps/                              # MLOps configurations and pipelines (e.g., MLflow, Kubeflow,  
DVC)  
        ├── mlflow_config.yaml  
        ├── kubeflow_pipelines/  
        └── dvc_config.yaml  
└─ datasets/                                # Versioned datasets for AI model training and evaluation  
(managed by DVC)  
    ├── raw/  
    ├── processed/  
    └── annotated/
```

This structure supports the entire MLOps lifecycle with a strong emphasis on open-source tools, from data preparation and model training to deployment and monitoring. It allows data scientists and ML engineers to work effectively within the larger platform context, leveraging the flexibility and community support of open-source AI.

6. `infra/` (Infrastructure as Code)

This directory defines the entire cloud infrastructure using Infrastructure as Code (IaC) tools. This ensures that the infrastructure is version-controlled, reproducible, and scalable.

```

skoola-platform/
└─ infra/
    ├── kubernetes/           # Kubernetes manifests (e.g., Helm charts, Kustomize)
    │   ├── apps/
    │   ├── services/
    │   └── ai/
    └── terraform/           # Terraform configurations for cloud resources (AWS, GCP,
Azure)
    ├── modules/
    ├── environments/
    │   ├── dev/
    │   ├── staging/
    │   └── prod/
    ├── main.tf
    ├── ansible/             # Ansible playbooks for configuration management
    │   ├── playbooks/
    │   └── roles/
    └── cloud-functions/      # Serverless functions (e.g., AWS Lambda, Google Cloud
Functions)
        ├── auth-triggers/
        └── payment-webhooks/

```

Separating infrastructure definitions from application code promotes a DevOps culture and enables automated provisioning and management of environments.

7. docs/ (Documentation)

This directory centralizes all project documentation, ensuring that information is easily accessible and up-to-date. This includes architectural decision records (ADRs), API documentation, design specifications, and operational guides.

```

skoola-platform/
└─ docs/
    ├── adrs/                 # Architectural Decision Records
    │   ├── 0001-choose-monorepo.md
    │   └── 0002-microservices-pattern.md
    ├── api/                  # OpenAPI/Swagger specifications for all services
    │   ├── auth-service.yaml
    │   └── course-management-service.yaml
    ├── design/               # UI/UX design specifications, wireframes, mockups
    │   ├── ux-ui-master-blueprint.md
    │   └── design-system-guidelines.md
    ├── operations/           # Deployment guides, monitoring runbooks, troubleshooting
    │   ├── deployment-guide.md
    │   └── monitoring-dashboard-setup.md
    ├── contributing.md       # Contribution guidelines for developers
    ├── README.md             # Documentation README
    └── glossary.md           # Project-specific terminology

```

Comprehensive documentation is vital for large, complex projects, facilitating knowledge sharing and reducing reliance on individual team members.

8. scripts/ (Helper Scripts)

This directory contains various helper scripts used for development, testing, CI/CD, and operational tasks. These scripts automate repetitive processes and standardize workflows.

```
skoola-platform/
└─ scripts/
   │ setup.sh           # Environment setup script
   │ build.sh           # Monorepo build script
   │ deploy.sh          # Deployment script
   │ test.sh            # Test runner script
   │ lint.sh            # Linting script
   │ db-migrate.sh      # Database migration script
   └─ start-dev-env.sh  # Script to start local development environment
```

Centralizing scripts makes them discoverable and ensures consistency in how tasks are performed across the team.

9. config/ (Global Configurations)

This directory stores global configuration files and templates that are shared across the platform, or define environment-specific settings.

```
skoola-platform/
└─ config/
   │ environments/      # Environment-specific configuration files
   │ │ development.json
   │ │ staging.json
   │ │ └─ production.json
   │ │ └─ default.json  # Default configuration values
   │ │ secrets.example  # Template for environment variables/secrets
   │ └─ logging.json    # Global logging configuration
```

Separating configurations from code allows for easier management of environment-specific settings and sensitive information.

10. data/ (Data Management)

This directory is for database-related assets, including schemas, migration scripts, and seed data.

```
skoola-platform/
└─ data/
   │ schemas/           # Database schemas (e.g., SQL DDL, Prisma schema)
   │ │ users.sql
   │ │ └─ courses.sql
   │ │ └─ migrations/   # Database migration scripts
   │ │ │ 20230101_initial_schema.sql
   │ │ │ └─ 20230215_add_my_stationery.sql
   │ │ └─ seed/
   │ │ │ initial_users.json # Seed data for development/testing
   │ │ │ └─ sample_courses.json
```

Centralizing database assets ensures consistency and simplifies database management across different environments.

11. tools/ (Development Tools)

This directory contains configurations for various development tools used across the monorepo.

```
skoola-platform/  
└─ tools/  
    ├── lint-staged.config.js # Configuration for lint-staged  
    ├── husky.config.js      # Configuration for Husky (git hooks)  
    ├── prettier.config.js   # Prettier formatting configuration  
    ├── eslintrc.js          # ESLint configuration  
    └─ jest.config.js        # Jest global test configuration
```

This ensures that all developers adhere to the same coding standards and tooling configurations.

12. tests/ (Platform-wide Tests)

This directory is for platform-wide tests that span across multiple services or applications, such as end-to-end (E2E) tests, integration tests, and performance tests.

```
skoola-platform/  
└─ tests/  
    ├── e2e/ # End-to-end tests (e.g., Playwright, Cypress)  
    │   ├── auth.spec.js  
    │   └─ course-flow.spec.js  
    ├── integration/ # Integration tests between services  
    │   ├── user-auth-integration.test.js  
    │   └─ payment-course-integration.test.js  
    └─ performance/ # Performance and load tests (e.g., k6, JMeter)  
        ├── load-test-scenario.js  
        └─ stress-test-scenario.js
```

Separating these tests from individual service/application tests provides a holistic view of the system's health and performance.

13. Conclusion

This comprehensive folder structure provides a robust and scalable foundation for the Skoola platform. By adopting a monorepo strategy with clearly defined directories for applications, services, shared libraries, AI components, and infrastructure, Skoola can ensure efficient development, seamless collaboration, and long-term maintainability. This blueprint is designed to evolve with the platform, accommodating future growth and technological advancements while maintaining a high standard of organization and clarity.

References

- [1] Reddit. *Choosing the Best Directory Structure for Scalable Microservice*. https://www.reddit.com/r/golang/comments/1eqx10i/a_comparison_choosing_the_best_directory/
- [2] Oso. *13 Microservices Best Practices*. <https://www.osohq.com/learn/microservices-best-practices>
- [3] Expert Soft. *Best Practices for Microservices Architecture*. <https://expert-soft.com/blog/best-practices-for-microservices-architecture/>
- [4] ResearchGate. (PDF) *Microservices Architecture for Large-Scale AI Applications*. https://www.researchgate.net/publication/387329307_Microservices_Architecture_for_Large-Scale_AI_Applications
- [5] Medium. *How to Build a Helm-Based Microservice Architecture with Centralized Charts*. <https://medium.com/codex/how-to-build-a-helm-based-microservice-architecture-with-centralized-charts-4de9250cfdec>
- [6] Dev.to. *Effective Project Structuring for Microservices with Quarkus*. <https://dev.to/yanev/effective-project-structuring-for-microservices-with-quarkus-1lf0>
- [7] PLANEKS. *Microservices Python Development: 10 Best Practices*. <https://www.planeks.net/microservices-development-best-practices/>
- [8] Kanerika. *How to Implement Microservices Architecture for Scalable Applications*. <https://kanerika.com/blogs/microservices-architecture/>
- [9] Readyssetcloud.io. *Best Practices for Building Serverless Microservices*. <https://www.readyssetcloud.io/blog/allen.helton/serverless-microservice-best-practices/>
- [10] GitHub. *Best Practices for Building a Microservice Architecture*. <https://github.com/katopz/best-practices/blob/master/best-practices-for-building-a-microservice-architecture.md>